САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №2

Реализация микросервисов

Выполнил:
Люсин Дмитрий
Группа: 3339

Проверил:
Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Опираясь на технический дизайн микросервисной архитектуры, разработанный ранее, реализовать разделение монолитного бэкенд-приложения на отдельные микросервисы. Каждый сервис должен отвечать за свою предметную область и иметь собственную базу данных. В результате необходимо получить работающее API приложения бронирования столиков в ресторанах, построенное на микросервисной архитектуре.

В качестве исходного приложения использовался ранее реализованный REST API для бронирования столиков в ресторанах.

Ход работы

1. Общая идея реализации

Монолитное приложение было разделено на несколько сервисов по предметным областям. Для клиента была оставлена единая точка входа - API Gateway. Внутренние сервисы не вызываются клиентом напрямую, а получают запросы через Gateway или через внутренние REST-запросы от других сервисов.

2. Структура микросервисного проекта

Проект реализован как monorepo, где в одной общей директории находятся все микросервисы:

- services/api-gateway
- services/auth-user-service
- services/catalog-service
- services/menu-service
- services/media-service
- services/booking-service
- services/review-service
- docs/gateway-openapi.yaml
- docs/interservice-openapi.yaml

3. Реализованные микросервисы

Микросервис	Порт	База данных	Назначение
API Gateway	3000	-	Единая точка входа для клиента и маршрутизация запросов
Auth & User Service	3001	auth_user.sqlite	Регистрация, вход, JWT, профиль пользователя

Catalog Service	3002	catalog.sqlite	Рестораны, кухни, столики, фильтрация ресторанов
Menu Service	3003	menu.sqlite	Категории меню и блюда
Media Service	3004	media.sqlite	Фотографии ресторанов
Booking Service	3005	booking.sqlite	Создание, просмотр и отмена бронирований
Review Service	3006	review.sqlite	Создание, просмотр, изменение и удаление отзывов

4. Принцип database-per-service

В работе был использован принцип database-per-service. Это означает, что каждый сервис владеет только своей базой данных и не обращается напрямую к таблицам другого сервиса. Если одному сервису нужны данные из другой предметной области, он делает внутренний REST-запрос к соответствующему сервису.

База данных	Сервис	Основные данные
auth_user.sqlite	Auth & User Service	Пользователи
catalog.sqlite	Catalog Service	Рестораны, кухни, столики
menu.sqlite	Menu Service	Категории меню, блюда
media.sqlite	Media Service	Фотографии ресторанов
booking.sqlite	Booking Service	Бронирования
review.sqlite	Review Service	Отзывы

5. Межсервисное взаимодействие

Основное взаимодействие между сервисами реализовано через REST-запросы. Клиент отправляет запрос в API Gateway, а Gateway перенаправляет его в нужный сервис. Внутренние запросы между сервисами защищены сервисным токеном x-service-token.

Пример создания бронирования:

- клиент отправляет POST /api/bookings в API Gateway;
- API Gateway перенаправляет запрос в Booking Service;
- Booking Service проверяет пользователя через Auth & User Service;
- Booking Service проверяет ресторан и столик через Catalog Service;
- Booking Service проверяет отсутствие конфликта по времени и создаёт бронирование в своей базе данных.

6. Основные endpoints для проверки

- GET /health - проверка работы API Gateway;
- GET /api-docs - Swagger-документация;
- POST /api/auth/login - вход пользователя;
- GET /api/restaurants - получение списка ресторанов;
- GET /api/restaurants/{restaurantId}/menu - получение меню ресторана;
- GET /api/restaurants/{restaurantId}/photos - получение фотографий ресторана;
- GET /api/restaurants/{restaurantId}/tables - получение столиков ресторана;
- POST /api/bookings - создание бронирования;
- GET /api/bookings/my - история бронирований пользователя;
- POST /api/restaurants/{restaurantId}/reviews - создание отзыва..

Вывод

В ходе лабораторной работы монолитное приложение бронирования столиков было разделено на микросервисы. Были реализованы отдельные сервисы для пользователей, каталога ресторанов, меню, фотографий, бронирований и отзывов, а также API Gateway как единая точка входа. Каждый сервис получил собственную базу данных, что соответствует принципу database-per-service.

В результате была получена рабочая микросервисная версия приложения. Сервисы взаимодействуют между собой через внутренние REST-запросы, а пользователь работает с системой через API Gateway. Такой подход позволяет разделить ответственность между частями приложения и упростить дальнейшее развитие отдельных компонентов.